

Rapport complet du projet — Chatbot Amendis

Document de référence pour la rédaction du rapport de stage

Ce document retrace **l'intégralité** du projet, du premier jour à la livraison : contexte, technologies (avec versions), chronologie détaillée, chaque décision d'architecture, chaque bug rencontré et sa correction, et l'historique des commits. Il est conçu pour que tu puisses rédiger ton rapport de stage sans rien oublier.

0. Fiche d'identité du projet

Titre	Conception et développement d'un chatbot d'assistance client
Entreprise cible	Amendis (distribution d'eau et d'électricité, assainissement — Tanger & Tétouan ; groupe Veolia)
Réalisé par	Diyae ZIYANI
Encadrant	Mr. Otman TRIATE
Type	Stage d'observation — Proof of Concept (PoC)
Durée	4 semaines (début 1er juillet 2026)
Dépôt Git	github.com/diyaeziyani10/chatbot
Objectif	Un assistant conversationnel qui répond aux questions clients 24h/24, à partir des informations officielles du site www.amendis.ma , sans halluciner

1. Le cahier des charges initial et son évolution

1.1 Ce que demandait le cahier des charges (validé au départ)

Une architecture **hybride** en trois briques : 1. Un moteur de **compréhension du langage (NLU)** avec **Rasa Open Source**. 2. Une **gestion de dialogue déterministe** pour les procédures sensibles, avec **simulation d'une base de données** clients en **SQLite**. 3. Un **système de secours RAG** (Retrieval-Augmented Generation) avec **LangChain + un LLM** (API cloud ou Ollama local), pour répondre aux questions hors scénario à partir de la **documentation interne** (PDF). 4. Un **front-end** via Rasa Shell ou **Streamlit**. Contraintes : **sécurité** (données clients traitées localement, jamais via le LLM), **zéro hallucination**, **4 semaines**.

1.2 Les adaptations décidées au fil du projet (points clés à défendre)

- **Adaptation n°1 (source des connaissances)** : au lieu de PDF internes, on utilise le contenu du **site public www.amendis.ma** (scrapé). Le second site, **www.amendisclient.ma** (espace client), s'est révélé être une **application JavaScript (Angular) derrière authentification** → non scrapable et contenant des **données personnelles**. Il ne sert donc que de **cible de redirection** (« payez sur votre espace client »).
- **Adaptation n°2 (abandon de SQLite)** : voir §4. Décision d'aller vers une architecture **100 % documentaire**.
- **Adaptation n°3 (abandon de Rasa)** : voir §11. En fin de projet, Rasa a été **entièrement retiré**, le RAG assurant seul tout le travail.

⚠ **Ces trois adaptations sont des écarts au cahier des charges validé** (SQLite et Rasa y étaient des livrables). Elles sont **techniquement justifiées** mais doivent être **présentées à l'encadrant** et documentées comme choix d'ingénierie assumés.

2. Les technologies utilisées (précises)

2.1 Langage & environnement

- **Python 3.10.11** (choix imposé : Rasa 3.6 n'accepte que Python 3.8 à 3.10 ; pas 3.11+).
- **Deux environnements virtuels** au départ (`venv_rasa` , `venv_rag`) car **conflit de dépendance** : Rasa exige `pydantic` v1, LangChain exige `pydantic` v2 → incompatibles dans le même environnement. Après le retrait de Rasa, **un seul venv** (`venv_rag`) suffit.
- **Système** : Windows 11, shell PowerShell.
- **Outils** : `winget` (installation de Python et d'Ollama), **Git/GitHub**, **Edge headless** (génération de PDF de documentation).

2.2 Briques du chatbot (état final)

Techno	Version / modèle	Rôle
FastAPI + uvicorn	—	Service RAG (backend, port 8000)
Streamlit	—	Front-end de chat (port 8501)
LangChain	1.3.11 (+ <code>langchain-community</code> , <code>langchain-classic</code> 1.0.8, <code>langchain-text-splitters</code> , <code>langchain-huggingface</code> , <code>langchain-ollama</code> , <code>langchain-groq</code>)	Orchestration du RAG
ChromaDB	—	Base vectorielle (stockage des embeddings)
sentence-transformers	modèle <code>paraphrase-multilingual-MiniLM-L12-v2</code> (vecteurs de 384 dimensions, multilingue, ~470 Mo, exécuté en local)	Embeddings
rank_bm25	—	Recherche par mots-clés (BM25)
requests, BeautifulSoup4, lxml	—	Scraping de <code>www.amendis.ma</code>
python-dotenv	—	Chargement de la clé API depuis <code>.env</code>
Groq (API cloud)	<code>llama-3.3-70b-versatile</code> (principal), <code>llama-3.1-8b-instant</code> (secours)	Génération des réponses (LLM)
Ollama (local)	<code>llama3.2</code> (~2 Go)	LLM de dernier secours (hors-ligne)

2.3 Technologies utilisées puis RETIRÉES (à mentionner dans l'historique)

- **Rasa Open Source 3.6.21 + rasa-sdk 3.6.2** : moteur NLU (DIETClassifier) + gestion de dialogue. Retiré en fin de projet.
 - **SQLite** : simulation de la base clients. Abandonné (architecture 100 % documentaire).
-

3. Architecture — évolution en trois grandes versions

Version 1 — Hybride complète (Rasa + SQLite + RAG)

```
Streamlit → Rasa (5005) → serveur d'actions (5055) → SQLite (données clients)
                                     → RAG FastAPI (8000) → Ollama
```

- Rasa reconnaît 8 intentions, extrait le n° de contrat, interroge SQLite pour les factures, et bascule vers le RAG pour les questions hors scénario.

Version 2 — 100 % documentaire (Rasa minimal, sans SQLite)

```
Streamlit → Rasa (5005) → serveur d'actions (5055) → RAG FastAPI (8000) → Groq/Ollama
```

- Rasa ne gère plus que 3 politesses + une intention « panier » `question_documentaire` + le fallback. Tout le métier part au RAG. SQLite supprimée.

Version 3 — RAG pur (Rasa retiré) — VERSION LIVRÉE

```
Streamlit (8501) → RAG FastAPI (8000) → Groq (70b → 8b) → Ollama
                        |
                        |— ChromaDB (recherche hybride BM25 + vectorielle)
                        |— mémoire/ (par utilisateur)
```

- **2 services seulement.** Le RAG assure tout : politesses (filtre mots-clés), compréhension, recherche, mémoire, génération en streaming.

4. Chronologie détaillée (phase par phase)

Phase 0 — Cadrage et vérification de faisabilité

- Lecture du cahier des charges. Décision d'utiliser les deux sites Amendis comme source.
- **Vérification des sites** (avec l'outil de récupération web) :
- `www.amendis.ma/fr` : HTML **rendu côté serveur**, scrapable, FAQ riche (lecture de facture, calcul de consommation, résiliation...). → **source du corpus RAG**.
- `www.amendisclient.ma` : **SPA Angular**, HTML quasi vide, contenu généré côté client **après login**, données personnelles. → **non scrapable** ; devient cible de redirection.
- **Conclusion** : amendisclient.ma est exactement ce que la SQLite devait *simuler* (données clients).

Phase 1 — Mise en place de l'environnement

- **Aucun Python installé** sur la machine. Installation de **Python 3.10.11** via winget.
- **BUG n°1 — Disque plein** : le disque C: n'avait que **0,2 Go libres** sur 237 → `pip install` échoue avec `OSError: [Errno 28] No space left on device`. Après libération d'espace (~15 Go), réinstallation OK.
- Création des **deux venvs** et installation des dépendances (Rasa d'un côté, LangChain/RAG de l'autre).
- Découverte et documentation du **conflit pydantic v1/v2** → justification des deux venvs et du RAG en micro-service HTTP séparé.

Phase 2 — Première version (Rasa + SQLite + RAG)

Fichiers Rasa créés : - `config.yml` : pipeline NLU (WhitespaceTokenizer → RegexFeaturizer → RegexEntityExtractor → LexicalSyntacticFeaturizer → 2× CountVectorsFeaturizer dont un en *char n-grams 1-4* pour la **tolérance aux fautes** → DIETClassifier 100 époques → FallbackClassifier **seuil 0,6**) + politiques (Memoization, Rule, TED). - `domain.yml` : **8 intentions** (saluer, au_revoir, remercier, consulter_facture, payer_facture, signaler_fuite, donner_numero_contrat, nlu_fallback), l'entité + slot `numero_contrat`, le formulaire `facture_form`, les réponses, 4 actions. - `data/nlu.yml` (exemples FR + regex `5\d{5}` pour le n° de contrat), `data/rules.yml`, `data/stories.yml`. - `actions/actions.py` : 4 classes — `ValidateFactureForm`, `ActionConsulterFacture` (requête SQL **paramétrée**, anti-injection), `ActionSignalerFuite`, `ActionRagFallback`. - `endpoints.yml`, `credentials.yml`.

Base de données : - `database/init_db.py` : 3 tables (`clients`, `factures`, `incidents`), **5 clients** et **7 factures** fictifs.

RAG : - `scraper.py` : crawler de www.amendis.ma (pages graines FAQ/accueil, liens internes `/fr`, **max 60 pages**, 1 req/s). - `ingest.py` : découpage en fragments **800 caractères / chevauchement 120**, embeddings `paraphrase-multilingual-MiniLM-L12-v2`, stockage ChromaDB. - `rag_api.py` : FastAPI `/ask`, recherche **MMR**, prompt strict anti-hallucination, LLM **Ollama llama3.2**. - `frontend/app.py` : chat Streamlit ↔ API REST Rasa. - **PDF de documentation** généré (Edge headless) : `docs/Documentation-Chatbot-Amendis.pdf`.

Bugs de cette phase : - **BUG n°2 — Scraper** `UnicodeEncodeError: latin-1` : le `User-Agent` contenait un tiret long (« — »), caractère non-ASCII interdit dans un **en-tête HTTP** (les en-têtes doivent être en latin-1/ASCII). Corrigé (tiret simple). - **BUG n°3 —** `charmap codec can't encode` → : la console Windows (cp1252) ne peut pas afficher la flèche « → » utilisée dans un `print`. Contourné avec `PYTHONIOENCODING=utf-8`. - **BUG n°4 — Télémétrie Rasa** `charmap` : erreur bénigne au 1er lancement, se désactive seule. - Entraînement Rasa : **100 % de précision** (`i_acc=1`, `e_f1=1`) — normal avec peu d'exemples bien séparés (modèle « trop sûr de lui »).

Phase 3 — Apprentissage (mode tuteur)

- Choix pédagogique : **comprendre** Rasa plutôt que générer du code non maîtrisé (objectif soutenance).
 - Notions acquises : la **boucle Rasa** (message → NLU → policy → action → réponse) ; `domain.yml` = inventaire, `rules.yml` = câblage, `nlu.yml` = exemples ; le **pipeline NLU** ; les **politiques**.
 - Test en direct avec `rasa shell nlu` :
 - « bonjour » → `saluer` à 99,99 %.
 - « je voudrais régler ce que je dois » (jamais vu) → reconnu à 99,7 %, **mais classé** `consulter_facture` (à cause du mot « dois » présent dans les exemples de cette intention) alors que le sens était « payer ».
- Leçon clé : une confiance élevée ≠ une bonne réponse** ; le classificateur reconnaît ce qui *ressemble* à ses exemples.

Phase 4 — Décision d'architecture : 100 % documentaire (sans SQLite)

- **Constat** : les données personnelles (facture d'un client) ne peuvent **pas** venir du site (login + données privées + interdiction de les envoyer au LLM). La SQLite ne fait que les *simuler*.
- **Décision de Diyae** : passer en **100 % documentaire** — toutes les intentions répondent depuis amendis.ma via le RAG ; pour « ma facture », le bot explique la démarche et redirige vers l'espace client.
- **Ce qui a été supprimé** : dossier `database/`, `facture_form`, entité/slot `numero_contrat`, `RegexEntityExtractor`, `EntitySynonymMapper`, les actions SQLite, les intentions `consulter_facture` / `payer_facture` / `signaler_fuite` / `donner_numero_contrat`.
- **Ce qui reste dans Rasa** : 3 politesses + intention « panier » `question_documentaire` + `nlu_fallback`. **Seuil du FallbackClassifier monté de 0,6 à 0,75.**

- **BUG/PIÈGE n°5 — le softmax** : « comment payer ma facture » était classé `remercier` avec **0,94** de confiance ! Un classificateur softmax **répartit 100 % de sa confiance entre les classes connues** → une question métier peut être classée « politesse » avec une confiance ÉLEVÉE, donc le seuil de fallback (qui ne protège que des confiances BASSES) ne la rattrape pas. **Solution** : l'intention « panier » `question_documentaire` qui capte tout le métier et le route vers le RAG.

Phase 5 — Travail multi-PC & RAG de bout en bout

- **Passation entre PC** : création de `REPRISE-PROJET.md` (contexte complet). Le projet est sur GitHub.
- **Point important** : Git ne transporte **pas** les venvs (chemins absolus codés en dur → cassés si copiés) ni les fichiers générés (modèle Rasa, corpus, base ChromaDB) — ils sont **régénérés** sur chaque PC via des commandes. Seul le **code source** est versionné.
- Transformation documentaire réalisée (commit `cbdd750`).
- Sur la machine finale : `git pull`, réentraînement Rasa, **re-vectorisation** → **236 fragments**, installation d'**Ollama + llama3.2**.
- Améliorations d'`ingest.py` : filtrage du **bruit de navigation** (`NOISE_LINES`) et **contextualisation** (chaque fragment préfixé par `[Page Amendis : <titre>]`).

Phase 6 — Premiers bugs de fonctionnement & passage à Groq

- **BUG n°6 — Timeout Ollama** : le tout premier message échouait (« service momentanément indisponible ») car le **chargement à froid** des 2 Go de llama3.2 dépassait le `timeout=60` de `actions.py`. Corrigé → 180 s (et le front Streamlit → 200 s). *Leçon : chaque maillon amont doit être plus patient que le maillon aval.*
- **BUG n°7 — Lenteur (~2 minutes/réponse)** : diagnostic mesuré → recherche ~1 s, **génération LLM = le goulot** (Ollama sur CPU, machine à **7,7 Go de RAM**). Aucune optimisation locale ne pouvait atteindre 5-10 s.
- **Passage à Groq** (API cloud gratuite) : modèle `llama-3.3-70b-versatile` → réponses en **~1-2 s**, qualité en hausse (modèle ~20× plus gros). **Clé API** dans `rag_service/.env` (**non versionné**), repli automatique sur Ollama. Conforme au cahier des charges (« API cloud ou Ollama »).
- **BUG n°8 — Prompt trop prudent** : le bot répondait « je n'ai pas trouvé » même avec les bons extraits sous les yeux (petit modèle appliquant trop strictement la règle refuge). Corrigé : **mission positive d'abord**, phrase refuge en dernier recours.

Phase 7 — Interface professionnelle (design)

- Design conçu dans **Claude Design** (projet « Amendis Chatbot Landing »), option « **1b · Marketing + widget** » (landing page + widget de chat fonctionnel intégré).
- Couleurs **extraites de la vraie feuille de style d'amendis.ma** ; couleur signature retenue : **rouge Amendis/Veolia** `#E2001A`. Polices **Space Grotesk + Public Sans**. Thème dans `.streamlit/config.toml`.
- Logos : **Amendis** (récupéré du site) et **Veolia** (récupéré du projet Design). Placés dans `frontend/assets/`.
- **BUG n°9 — Mauvais logo** : le fichier `2.png.webp` téléchargé du site était en réalité un **logo Veolia**, pas Amendis → l'en-tête affichait « opéré par VEOLIA » à gauche. Corrigé en téléchargeant le vrai logo Amendis (`Amendis_1.jpeg`, 335×152).
- Améliorations UX successives :
- suppression du message « Press Enter to submit form » (CSS).
- la **question de l'utilisateur s'affiche instantanément** + indicateur « ... » animé, **avant** la réponse (séparation en deux temps).

- **barre de défilement interne** au chat (astuce CSS `flex-direction: column-reverse` → le fil reste collé en bas, la page ne s'allonge plus).

Phase 8 — « Intelligence » (remarques de l'encadrant)

L'encadrant juge le bot « **simple moteur de recherche** », sans intelligence. Tests précis qu'il a faits : - « donne-moi les documents pour un abonnement à **Maroc Telecom** » → le bot **cherchait** dans Amendis et répondait « pas trouvé » au lieu de **comprendre** que c'est hors domaine. - « **qui es-tu ?** » → le bot présentait *Amendis* (la page « à propos ») au lieu de se présenter *lui-même*. - Attente d'une **mémoire** (une question posée hier, reposée aujourd'hui).

Solutions apportées (commit `fin2 = aaf2acc`) : 1. **Prompt « persona »** (contrat de comportement) avec des **cas de réaction** numérotés : question Amendis → extraits ; « qui es-tu ? » → identité ; hors-domaine → refus **sans donner d'info** + rebond vers un service Amendis ; info absente → aveu honnête ; suivi de conversation → historique ; ambiguïté → clarification ; **proactivité** (une suggestion en fin de réponse). 2. **Mémoire persistante par utilisateur** : un fichier JSON par utilisateur dans `rag_service/memoire/` (identifiant assaini par regex, **50 échanges conservés, 6 injectés** dans le prompt). Champ `🇲🇦 « votre nom / n° de client »` dans le front (sinon UUID anonyme). Données personnelles **hors Git**. 3. **Reformulation de question** (« condense question ») : quand un historique existe, un 1er appel LLM transforme un message contextuel (« oui », « et pour l'électricité ? ») en **question autonome** avant la recherche. 4. **Chaîne de secours LLM à 3 niveaux** (voir bug n°10).

Bugs découverts pendant cette phase : - **BUG n°10 — Quota Groq épuisé (429)** : le quota gratuit du 70b est de **100 000 tokens/jour** (~20-30 questions). En pleine séance de tests → panne. **Solution** : chaîne de secours `LLMs` = Groq 70b → Groq 8b-instant (quota séparé) → Ollama local. La **reformulation** (tâche simple) est confiée au 8b pour économiser le 70b. - **BUG n°11 — « qui est Netflix » à moitié refusé** : le bot refusait *mais expliquait quand même* ce qu'est Netflix. **Corrigé** : règle durcie « refuse SANS donner la moindre information, même si tu connais la réponse ». - **BUG n°12 — « offres de travail d'Amendis » : contradiction** : le bot disait « je ne peux pas répondre » puis décrivait la page carrières. Cause : le domaine était défini trop étroitement (« eau et électricité »). **Corrigé** : domaine élargi à *tout ce qui concerne Amendis* (carrières incluses) + consigne d'**interprétation d'intention**. - Plan de test manuel formalisé : `TESTS.md` (12 sections).

Phase 9 — Bugs restants, agences, hallucinations

- **BUG n°13 — Charabia classé politesse** : « iam », « asdnsabdi » → classés `saluer / remercier`. Ajout d'une intention `incompris` (exemples de charabia) + réentraînement. Après le retrait de Rasa, ce cas est géré par une **règle du prompt** (le LLM demande de reformuler).
- **BUG n°14 — « oui » classé `saluer`** : après une suggestion du bot, répondre « oui » donnait « Bonjour ! ». Ajout de **12 exemples d'affirmations** (« oui », « d'accord », « je veux les deux »...) à `question_documentaire` + réentraînement → « oui » classé à 100 %. Complété par la reformulation (« oui » → question complète).
- **Agences absentes du corpus** : le site liste les agences dans une **carte JavaScript** non scrapée. Solution : fichier **manuel** `corpus/fr_nos-agences.txt` (une trentaine d'agences avec adresses), **versionné** via une exception dans `.gitignore`. Découpage **une agence par fragment** (sinon un nom précis « se noie » dans la liste).
- **BUG n°15 — Hallucination sur les agences** : « et les agences de Fnideq » → le bot **inventait** une adresse (« Rue Ibn Khaldoun » au lieu de la vraie), un **nom d'agence inexistant** (« Agence Al Massira », en fait une adresse d'une autre agence), et des **horaires** absents du corpus. **Corrigé** : règles anti-hallucination durcies — « recopie les données factuelles à l'identique, ne combine jamais deux agences, n'invente ni adresse ni horaire ». Résultat : le bot **avoue** plutôt qu'il n'invente.

- **BUG n°16 — Réponses « paresseuses »** : le bot donnait quelques agences puis renvoyait au site. Tentative de correction (« sois exhaustif, ne renvoie jamais au site » + injection de la liste complète) → **effet secondaire : réponses plus lentes et plus de fautes → ANNULÉ** (retour à l'état concis).

Phase 10 — Recherche hybride (BM25 + vectorielle)

- Ajout de **BM25** (recherche par mots-clés) combiné à la recherche vectorielle via `EnsembleRetriever` (fusion RRF, poids 0,5/0,5, k=6 chacun). Bibliothèques : `rank_bm25`, `langchain-classic`.
- La fonction `build_chunks()` est **partagée** entre `ingest.py` (pour ChromaDB) et `rag_api.py` (pour BM25).
- **BUG n°17 — Import déplacé** : `from langchain.retrievers import EnsembleRetriever` échouait (LangChain 1.3 a déplacé la classe vers `langchain_classic.retrievers`). Corrigé.
- **Gain majeur** : les noms propres et chiffres (agences, numéros) que l'embedding ratait sont désormais rattrapés par les mots-clés. « quelles agences à Tétouan ? » liste enfin les vraies agences.

Phase 11 — Retrait complet de Rasa

- **Constat** : Diyae a **proposé Rasa elle-même** (pas imposé) ; son rôle était devenu résiduel (3 politesses + routage). Retrait décidé pour la **latence** (~3,7 s de Rasa mesurés sur les sauts HTTP + inférence TensorFlow CPU) et la **simplicité**.
- **Réalisé** : nouveaux endpoints `/chat` et `/chat_stream` dans `rag_api.py`, **filtre de politesses** par mots-clés (réponse fixe instantanée, sans LLM), charabia géré par le prompt. Le front appelle **directement** le RAG. **Passage de 4 services à 2.** (commit `4c87067`.)

Phase 12 — Réponse en streaming

- **Backend** : `generer_stream()` (streaming via la chaîne de secours) + endpoint `/chat_stream` (`StreamingResponse`). `max_retries=0` sur Groq (échec rapide, plus de réessais lents ~5-8 s).
- **Frontend** : affichage token par token via un **placeholder** ; **décodeur UTF-8 incrémental** pour ne pas casser les accents à cheval sur deux morceaux réseau.
- **Résultat mesuré** : **premier mot en ~1 s** → ressenti « quasi instantané ».
- **BUG n°18 — Pics à 18 s** : diagnostic → repli sur **Ollama** (CPU) déclenché par une limite Groq *par minute* + les réessais internes du client Groq. Atténué par `max_retries=0`.
- **Note** : le modèle `openai/gpt-oss-120b` a été envisagé (raisonnement), **testé et jugé décevant** (verbeux, rigide) → on est **resté sur llama-3.3-70b-versatile**.

Phase 13 — Nettoyage (code livrable)

- Suppression de : `rasa_bot/`, `database/`, dossier dupliqué `chatbot/`, `requirements-rasa.txt`, et l'environnement `venv_rasa`.
- `.gitignore` nettoyé, docstrings/commentaires actualisés (plus aucune référence à Rasa/SQLite). (commit `fc35056`.)

Phase 14 — Pistes d'amélioration (recherche, non implémentées)

Étudiées mais **volontairement non intégrées** (deadline / risque) — à mettre en **perspectives d'évolution** du rapport : - **Évaluation systématique (RAGAS)** : métriques *faithfulness* (fidélité = mesure de l'hallucination), *context precision/recall*, *answer relevancy*. - **Re-ranking** : 2^e étape qui re-classe les fragments avec un **cross-encodeur** (lit question + fragment ensemble, plus précis que les embeddings « bi-encodeur »). Gain +5 à +15 points de pertinence. Via API (Cohere/Jina, ~zéro RAM) ou local (BGE, coûteux en RAM). - **RAG-Fusion** : plusieurs reformulations de la question + fusion RRF (extension naturelle de l'EnsembleRetriever déjà en place).

- **Meilleurs embeddings** (bge-m3, ~2,3 Go) : **bloqué** par la RAM (seulement ~0,6 Go libres sur la machine). - **Cache sémantique** : mémoriser les paires question/réponse et répondre instantanément à une question déjà posée (le vrai sens de « bot qui apprend / répond plus vite aux répétitions » — **aucun LLM ne s'entraîne tout seul en direct**). - Projets open-source de référence : `langchain-ai/rag-from-scratch` (techniques), `explodinggradients/ragas` (évaluation), `infiniflow/ragflow`, `weaviate/Verba`.

5. Architecture finale livrée — fonctionnement détaillé

5.1 Le parcours d'un message

1. Le **front** envoie `{question, user_id}` à `POST /chat_stream`.
2. **Filtre politesses** : si « bonjour/merci/au revoir... » → réponse fixe **instantanée, sans LLM**.
3. **Mémoire** : lecture des 6 derniers échanges de l'utilisateur.
4. **Reformulation** : si la question dépend du contexte, un 1er LLM la rend autonome.
5. **Recherche hybride** : BM25 (mots) + vectorielle MMR (sens), fusionnées par RRF → fragments pertinents.
6. **Génération en streaming** : le LLM (Groq 70b, sinon 8b, sinon Ollama) rédige la réponse **à partir des fragments uniquement**, token par token.
7. **Front** : affichage en direct.
8. **Mémoire** : la paire question/réponse est enregistrée.

5.2 Fichier par fichier (état livré)

- `rag_service/scrapper.py` — crawler de `www.amendis.ma` → `corpus/*.txt`.
- `rag_service/ingest.py` — `build_chunks()` (découpage + contextualisation, agences ligne par ligne) + `main()` (embeddings → ChromaDB).
- `rag_service/rag_api.py` — **le cœur** : mémoire, prompt-contrat, recherche hybride, chaîne de secours LLM, reformulation, endpoints `/chat`, `/chat_stream`, `/health`.
- `frontend/app.py` — interface Streamlit (design widget, streaming, champ identité, thème rouge).
- `rag_service/corpus/` — texte du site + `fr_nos-agences.txt` (manuel, versionné).
- `rag_service/chroma_db/`, `rag_service/memoire/`, `rag_service/.env` — générés/secrets, **non versionnés** (sauf agences).
- `.streamlit/config.toml` — thème. `docs/` — documentation. `README.md`, `TESTS.md`.

5.3 Chiffres-clés

- **61 pages** scrapées + fichier agences → **267 fragments** dans ChromaDB.
 - Vecteurs de **384** dimensions. Recherche : **6 fragments** BM25 + 6 vectoriels fusionnés.
 - Mémoire : **50** échanges conservés, **6** injectés. Seuil (ex-Rasa) : 0,75.
 - Latence : **~1 s** (premier mot en streaming) ; politesses instantanées.
 - Quota Groq 70b : **100 000 tokens/jour**.
-

6. Journal complet des erreurs & corrections (récapitulatif)

#	Erreur / bug	Cause	Correction
1	<code>pip</code> « No space left on device »	Disque C: plein (0,2 Go)	Libérer ~15 Go
2	Scraper <code>UnicodeEncodeError latin-1</code>	Tiret long non-ASCII dans le User-Agent (en-tête HTTP)	User-Agent en ASCII
3	<code>charmap can't encode →</code>	Console Windows cp1252 vs « → »	<code>PYTHONIOENCODING=utf-8</code>
4	Télémetrie Rasa <code>charmap</code>	Console Windows	Bénin, ignoré
5	« payer facture » → <code>remercier</code> 0,94	Piège du softmax	Intention « panier » <code>question_documentaire</code>
6	Timeout « service indisponible »	Chargement à froid Ollama > 60 s	timeout 180/200 s
7	~2 min/réponse	Ollama sur CPU (8 Go RAM)	Passage à Groq (cloud)
8	Refus « pas trouvé » à tort	Prompt trop prudent	Mission positive d'abord
9	Logo Veolia à la place d'Amendis	Mauvais fichier téléchargé	Vrai logo Amendis
10	Panne quota Groq (429)	100k tokens/jour épuisés	Chaîne de secours 70b→8b→Ollama
11	« qui est Netflix » à moitié refusé	Règle hors-domaine trop molle	Refus SANS aucune info
12	« offres travail Amendis » contradictoire	Domaine trop étroit	Domaine élargi + interprétation
13	Charabia → politesse	Softmax	Intention <code>incompris</code> (puis prompt)
14	« oui » → <code>saluer</code>	NLU sans affirmations	12 exemples + reformulation
15	Adresses d'agences inventées	LLM combine des fragments	Anti-hallucination durci
16	Réponses lentes/verbeuses	Règle « exhaustif » + injection	Annulé (retour concis)
17	Import <code>EnsembleRetriever</code> KO	Déplacé dans LangChain 1.3	<code>langchain_classic.retrievers</code>
18	Pics de latence 18 s	Repli Ollama + réessais Groq	<code>max_retries=0</code>

7. Historique des commits Git (du plus ancien au plus récent)

- `572634b` — tt
- `63eb2dd` — transfert
- `cbdd750` — Transformation architecture 100 % documentaire (suppression SQLite)
- `d64d5cb` — Mise à jour du fichier de passation (fin session PC n°2)
- `edca8bf` — **fin?** (projet initial complet + PDF de documentation)
- `aaf2acc` — **fin2** (intelligence : persona, mémoire, reformulation, secours LLM)
- `a4e747d` — Intelligence conversationnelle + agences (+ anti-hallucination)
- `4c87067` — Retrait de Rasa + recherche hybride + streaming
- `fc35056` — Nettoyage : suppression complète de Rasa + code livrable

8. Réponse aux contraintes du cahier des charges

- **Sécurité des données**  : aucune donnée client dans le système (architecture 100 % documentaire) ; seuls la question et des extraits du site **public** partent vers le LLM. Mémoire conversationnelle **locale**

uniquement.

- **Zéro hallucination** ✅ : le prompt impose de répondre uniquement à partir des extraits, de recopier les faits à l'identique, et d'avouer quand l'info manque (température 0).
- **Délai 4 semaines** ✅ : PoC fonctionnel livré.
- **Déploiement optionnel** : le service RAG expose une API HTTP → intégrable à un site officiel (le front Streamlit remplaçable par un widget).

9. Limites connues (à assumer dans le rapport)

- **Latence** ~1-8 s selon la charge Groq ; pics possibles si repli sur Ollama (CPU).
- **Couverture** limitée aux 61 pages scrapées ; certaines infos (tarifs détaillés, agences en carte JS) manquent ou ont été ajoutées à la main.
- Le charabia **très court** reste parfois ambigu.
- Pas d'**évaluation quantitative** systématique (tests manuels via `TESTS.md`).
- **Embeddings** limités par la RAM (MiniLM au lieu de bge-m3).

10. Perspectives d'évolution (pour le rapport)

Évaluation automatisée (RAGAS) · Re-ranking (cross-encodeur) · RAG-Fusion · Cache sémantique (réponses instantanées aux questions répétées) · Meilleurs embeddings sur une machine plus puissante · Déploiement serveur (latence < 1 s) · Widget intégré au site officiel · Retour de Rasa possible pour un vrai dialogue à étapes si de nouveaux scénarios métier apparaissent.

11. Glossaire (concepts à maîtriser en soutenance)

- **RAG (Retrieval-Augmented Generation)** : chercher les documents pertinents, puis faire rédiger le LLM à partir d'eux.
- **Embedding** : transformation d'un texte en vecteur de nombres capturant son *sens*.
- **Base vectorielle (ChromaDB)** : stocke les vecteurs et retrouve les plus proches d'une requête.
- **Recherche hybride** : vectorielle (sens) + BM25 (mots exacts), fusionnées par **RRF** (Reciprocal Rank Fusion).
- **Chunking** : découpage des documents en fragments.
- **Prompt engineering** : spécifier le comportement du LLM comme un contrat écrit.
- **Softmax** : un classificateur répartit 100 % de sa confiance entre les classes connues (d'où des confiances élevées trompeuses).
- **Bi-encodeur vs cross-encodeur** : embeddings (question et doc encodés séparément, rapide) vs reranker (ensemble, précis).
- **Streaming** : renvoi de la réponse token par token → ressenti quasi instantané.
- **Zéro hallucination** : le modèle ne répond que d'après les extraits fournis, jamais d'invention.
- **Chaîne de secours (fallback)** : plusieurs LLM en cascade pour ne jamais tomber en panne. ``